MLP vs. CNN Performance Evaluation

Ada Gabriela Tatulescu

Università di Trento

1 MLP Project: Introduction and Theoretical Context

The main objective of this study is to evaluate the generalization capability of a Multi-Layer Perceptron (MLP) in a binary image classification task (Dogs vs. Cats), using an unsupervised pre-training stage.

A key challenge in applying MLPs to computer vision is their sensitivity to pixel-level variations, as they are not designed to capture spatial structure. To address this limitation, data augmentation and unsupervised feature learning via an autoencoder were used.

1.1 Model Configuration and Regularization Strategies

Data Augmentation When using an MLP, the risk of overfitting is high, as the model may memorize pixel-level noise instead of learning meaningful patterns. To mitigate this issue, a train transform strategy was applied:

By using *RandomRotation(15)* and *RandomHorizontalFlip*, the model is trained to recognize that an animal (dog or cat) maintains its identity regardless of its orientation. This approach partially covers the lack of spatial invariance inherent in linear layers.

ColorJitter simulates real-world lighting variations and prevents the model from incorrectly associating a certain brightness or contrast with a class (dogs or cats).

Normalization using the *ImageNet* means and standard deviations aligns the input distribution with an already validated feature space.

Regarding *val_test_transform*, resizing and normalization are applied, but no data augmentation is performed. Evaluation is done on real images without distortions.

Input Dimensionality and Input Space A resolution of 64×64 pixels was chosen because a total input size of $64 \times 64 \times 3 = 12,288$ is computationally manageable to preserve details such as ear shape and the fur texture of dogs and cats, essential for distinguishing between the species.

Optimization Hyperparameters A *learning rate* of $1e-3$ was selected, representing the standard setting for the Adam optimizer, ensuring stability and fast convergence during network training. A *batch size* of 32 was chosen because experiments with smaller values revealed increased training noise. A *hidden dimension* of 64 balances effective feature learning, underfitting at smaller sizes, and increased computational cost at larger ones.

1.2 Data infrastructure

The *MyCustomDataModule* class ensures that the processing pipeline remains consistent regardless of the hardware used (GPU or CPU).

Preventing Data Leakage Because the dataset is split into three categories (train, validation, and test), there is a risk that data could be mixed or leaked between sets. Each subset was loaded using *ImageFolder*, which ensures that data from one subset was not used in another.

Data Augmentation on Training Data augmentation was applied only on the training set because the goal is for the model to see different images at each epoch and learn general features instead of memorizing the training images. The validation and test sets must contain real and consistent images in order to perform correctly on unseen data.

Shuffle for Training For the training set, a DataLoader with *shuffle=True* is used so that images are seen in a different order at each epoch, while the validation and test sets are loaded without shuffling to ensure a fair evaluation.

1.3 MLP Autoencoder

The main objective of the class *MLPAutoencoder* is to perform dimensionality reduction and feature extraction.

The architecture uses a symmetric bottleneck: the encoder compresses the flattened 12288-input into a 256-dimensional latent vector capturing key visual features, while the decoder reconstructs the input, preserving maximal information. The 256-dimensional latent space was selected empirically, as smaller sizes caused information loss, while larger ones led to pixel-level memorization.

To measure the pixel-level difference between the original and reconstructed images, the *Mean Squared Error* loss function is used. A low MSE indicates that the latent space provides a faithful representation of the original data.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2 \tag{1}$$

1.4 Neural Network

Neural Network Architecture for image classification The final model, *MLPWithPretrainedEncoder*, uses a transfer learning approach by combining a pretrained encoder for feature extraction with a simple classification head. The classification head has a two-layer MLP with a hidden dimension of 128 units and ReLU activation. The final layer outputs two logits corresponding to the Dog and Cat classes and allows the model to use the features learned during pretraining for the classification task.

Freezing Mechanism By setting *requires_grad = False*, we keep the spatial features learned in earlier steps and train only the classification part.

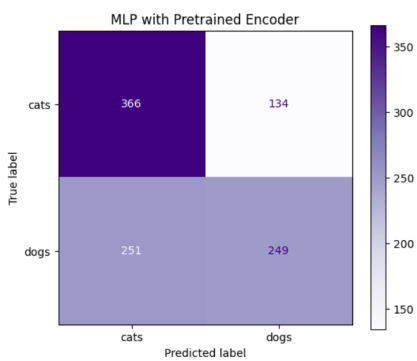
Observation Before applying the Autoencoder, the model was forced to learn complex visual features from a random initialization. This led to overfitting, and the task to find direct correlations between raw pixel intensities. The observed 2% accuracy boost confirms that the Autoencoder improves performance, together with Data Augmentation.

1.5 MLP Model With Autoencoder Results

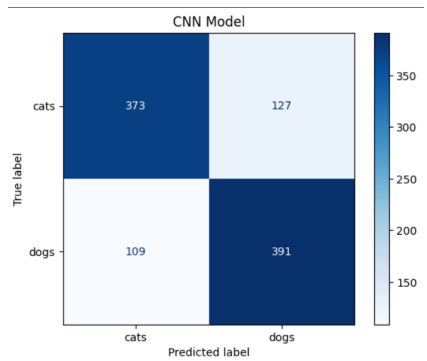
Analysis of Model Parameters and Complexity From the model summary table, we observe the following distribution: Encoder: 12.8 million parameters, Classifier: 33.2 thousand parameters, Total: 12.9 million parameters.

Training vs. Validation Performance, Final Performance The model achieves an accuracy of 56.2% on the training set and 62.5% on the validation set. Although the loss is slightly higher on the validation set (0.76 compared to 0.71), the learning process remains stable. The test loss is approximately 0.77, comparable to the training and validation losses, indicating stable model behavior. The fact that the test set accuracy (61.5%) is very close to the validation accuracy confirms good generalization ability, demonstrating that the model extracted relevant features without suffering from overfitting.

Confusion Matrix Interpretation for MLP Model The confusion matrix shows that the model performs better at recognizing cats (366 correct predictions), while performance on dogs is weaker, with nearly as many misclassifications as correct predictions (251 vs. 249). This asymmetry indicates a bias toward the cats class, with many dog samples being absorbed into it. The results suggest unbalanced feature learning and increased confusion for the dog category.



MLP Confusion Matrix



MLP Model Results

1.6 Implementing CNN

The implemented CNN architecture is significantly more efficient for processing 64×64 pixel images, as it learns hierarchical feature representations. The model is structured around a convolutional feature extraction module followed by a fully connected classification head.

Feature Extraction: The CNN model employs *nn.Conv2d* layers with 3×3 kernels. These convolutional filters operate locally on the image, enabling the network to automatically learn low-level features such as edges and texture.

Stability and Non-Linearity: Each convolutional layer is followed by a *BatchNorm2d* layer, to stabilize training and speed up convergence. The ReLU activation function introduces non-linearity to model complex patterns in the data.

Spatial Downsampling: The inclusion of *MaxPool2d(2)* layers progressively reduces the spatial resolution of the feature maps. This process decreases computational complexity while preserving the most salient features.

Overfitting Mitigation: In the classification stage, a *Dropout(0.3)* layer is employed to randomly deactivate a subset of neurons during training. It reduces the risk of overfitting and improves the model's ability to generalize to unseen data.

1.7 CNN Model Results improvements

The CNN model shows a strong improvement in classification performance, achieving a test accuracy of 76.4%, which represents a substantial gain over the MLP approach.

The confusion matrix shows that the CNN correctly classifies most samples from both classes. For cats, 373 images are correctly identified, while 127 are misclassified as dogs. For dogs, 391 images are correctly classified, with 109 misclassified as cats. The model performs slightly better on the dogs class, and the results indicate good feature separability with moderate confusion between classes.

2 Bibliography

- Course materials Regression, Classification, Unsupervised Learning by Prof. Nicola Conci
- http://scikit-learn.org/
- https://www.geeksforgeeks.org/
- https://www.mygreatlearning.com/blog/introduction-to-support-vector-machine/